

✓ Plano de Ensino Resumido

[Contrair tudo](#)

Horários

Horário dos encontros: Quartas, das 8h15 às 9h45 e Sextas das 8h15 às 10h45

Horário oficial de atendimento: Quartas, das 13h30 às 14h15

Atendimento via call (Google Meet): Agendar via <https://wa.me/+5553981167534>, durante a semana.

Conheça a formação do seu professor

Link para o currículo na Plataforma Lattes: <http://lattes.cnpq.br/4733841203436666>

METODOLOGIA E RECURSOS UTILIZADOS

1. As atividades em sala de aula acontecerão nos formatos *Live Code* e Oral Expositivas, apoiadas por desenvolvimento de projeto para fixação.
2. As atividades para casa serão disponibilizadas no AVA Moodle e envolverão:

Leituras de documentações do Expo, React Native, Firebase e Javascript ES6;
Entrega do app desenvolvido no Github, em repositório e *branch* indicada pelo professor.

3. Ferramentas necessárias:

- Computador com, no mínimo, 8GB de memória RAM, e sistema operacional macOS, Windows (Powershell e [WSL 2](#)), e Linux suportado pelo NodeJS LTS;
- IDE VSCode (prefira a mais atual) e algumas extensões (listadas no item 4);
- NodeJS LTS;
- Git e Github;
- Github Copilot.

4. Extensões do VSCode (recomendadas)

- ESLint - formatador de código padrão das tecnologias js e jsx, e ts e tsx <https://marketplace.visualstudio.com/items?itemName=dbaeumer.vscode-eslint>
- Prettier - formatador de código que adiciona 'habilidades' ao ESLint <https://marketplace.visualstudio.com/items?itemName=esbenp.prettier-vscode>
- Color Highlight - ajuda o desenvolvedor a ver a cor que ele está adicionando a paleta de cores do projeto <https://marketplace.visualstudio.com/items?itemName=naumovs.color-highlight>
- Expo Tools (para Expo) - um Intellisense para Expo <https://marketplace.visualstudio.com/items?itemName=expo.vscode-expo-tools>
- Github Copilot - Nossa! Ajuda muito o desenvolvedor. Finalmente uma ferramenta que faz isso! <https://marketplace.visualstudio.com/items?itemName=GitHub.copilot>
- GitHub Copilot Chat - Se precisarmos detalhar as sugestões do Copilot isso ajuda muito <https://marketplace.visualstudio.com/items?itemName=GitHub.copilot-chat>
- Reactjs code snippets - um Intellisense com dicas para o React
- React Native Tools - um Intellisense para React Native <https://marketplace.visualstudio.com/items?itemName=msjsdiag.vscode-react-native>
- React-Native/React/Redux snippets for es6/es7 - um Intellisense para Redux <https://marketplace.visualstudio.com/items?itemName=EQuimper.react-native-react-redux>
- TODO Highlight - escaneia todos os comentários com todo (tarefas por ainda fazer no código) <https://marketplace.visualstudio.com/items?itemName=wayou.vscode-todo-highlight>
- vscode-icons - um grupo de ícones para ajudar nos achar mais rápido no visual dos arquivos e pastas do VSCode <https://marketplace.visualstudio.com/items?itemName=vscode-icons-team.vscode-icons>
- React Native Paper Snippets - um Intellisense com dicas para a lib de UI React Native Paper <https://marketplace.visualstudio.com/items?itemName=arpitBhalla.rnp-snippets>

5. Configurações do VSCode (recomendadas para https://code.visualstudio.com/Docs/languages/javascript#_code-actions-on-save)

- ESLint: analisa estaticamente o código para encontrar problemas de formatação e sintaxe. Ele pode ser integrado ao editor de código e ser executado como parte do seu pipeline de integração contínua. No VSCode busque por "formatOnSave" e marque "Editor: formatOnSave"
- Prettier: é um formatador de código que adiciona uma camada extra ao ESLint. No VSCode busque por "defaultFormatter" e marque "Editor: Default Formatter".

Nota: Nós utilizaremos o Prettier para questões de formatação de código e o ESLint para questões de qualidade de código (sintaxe).

METODOLOGIA DE AVALIAÇÃO E REAVALIAÇÃO

A avaliação se dará pelo atendimento dos objetivos listados a seguir:

1. Implementação do Módulo 1 (2,0) → Funcionalidade SignIn, SignUp, CRUD do Perfil do usuário, e afins
2. Implementação do Módulo 2 (1,5) → Funcionalidade Upload de imagens
3. Implementação do Módulo 3 (2,5) → Funcionalidade CRUD via SDK do Firestore Web
4. Implementação do Módulo 4 (2,0) → Funcionalidade Notificações
5. Implementação do Módulo 5 (2,0) → Funcionalidade Mapas e localização

Nota final (Peso 10,0) = Módulo 1 (2,0) + Módulo 2 (1,5) + Módulo 3 (2,5) + Módulo 4 (2,0) + Módulo 5 (2,0)

Todas as **avaliações e reavaliações são individuais** e devem ser entregues via Github em projeto único.

As **avaliações** serão realizadas conforme o cronograma previsto no **plano de ensino**. O aluno deve **defender** o módulo que implementou perante o professor.

A **reavaliação** acontecerá no mesmo formato da avaliação, conforme o cronograma previsto no **plano de ensino**.

Toda entrega que perder o **prazo**, conforme as datas previstas no cronograma do **plano de ensino**, sofrerá uma **penalidade de 20% na nota**.

REFERÊNCIAS

EXPO. **Introdução: Get started creating apps with Expo**. Disponível em: <https://docs.expo.dev/get-started/introduction/>. Acesso em março de 2025.

reactnative.dev/. Acesso março, 2025.

NPMJS. **Repositório de dependências Node Package Manager em JS ou TS**. Disponível em: <https://www.npmjs.com/>. Acesso março de 2025.

FIREBASE. **Firestore Docs**. Disponível em: <https://firebase.google.com/docs?hl=pt-br> Acesso março de 2025.

Avisos

Projeto de Exemplo no Github

✓ Concluído

Projeto desenvolvido em sala de aula (GitHub)

✓ Concluído

Link para a documentação oficial do React Native

✓ Concluído

Link para a documentação oficial do Expo

✓ Concluído

Plano de Ensino

✓ Concluído

✓ Semana 1 - Ambiente de Desenvolvimento e Fundamentos de React - 05/08/2026 e 07/08/2026

Em aula

- Apresentação do [plano de ensino](#)
- Configuração do ambiente na plataforma Linux
- Criando o primeiro app
- Conhecendo o ambiente de desenvolvimento

Para casa

1. **Criar o primeiro app**. Siga os seguintes passos:

1.1 - **Configure o ambiente para Expo**: <https://docs.expo.dev/get-started/create-a-project/>

Algumas dicas:

1.1.1 - Instale o **npm** (<https://github.com/npm-sh/nvm>) e instale ou atualize o **NodeJS** para a última versão **LTS** (<https://nodejs.org/en/>);

1.2 **Crie o seu primeiro projeto Expo:** `npx create-expo-app@latest` (faça o nome do projeto ter esse formato `pdm_seuNomeAqui`) <https://docs.expo.dev/get-started/create-a-project/>

2. Instale as extensões do VSCode (recomendadas)

- ESLint - formatador de código padrão das tecnologias js e jsx, e ts e tsx <https://marketplace.visualstudio.com/items?itemName=dbaeumer.vscode-eslint>
- Prettier - formatador de código que adiciona 'habilidades' ao ESLint <https://marketplace.visualstudio.com/items?itemName=esbenp.prettier-vscode>
- Color Highlight - ajuda o desenvolvedor a ver a cor que ele está adicionando a paleta de cores do projeto <https://marketplace.visualstudio.com/items?itemName=naumovs.color-highlight>
- Expo Tools (para Expo) - um Intellisense para Expo <https://marketplace.visualstudio.com/items?itemName=expo.vscode-expo-tools>
- Github Copilot - Nossa! Ajuda muito o desenvolvedor. Finalmente uma ferramenta que faz isso! <https://marketplace.visualstudio.com/items?itemName=GitHub.copilot>
- GitHub Copilot Chat - Se precisarmos detalhar as sugestões do Copilot isso ajuda muito <https://marketplace.visualstudio.com/items?itemName=GitHub.copilot-chat>
- Reactjs code snippets - um Intellisense com dicas para o React
- React Native Tools - um Intellisense para React Native <https://marketplace.visualstudio.com/items?itemName=msjsdiag.vscode-react-native>
- React-Native/React/Redux snippets for es6/es7 - um Intellisense para Redux <https://marketplace.visualstudio.com/items?itemName=EQuimper.react-native-react-redux>
- TODO Highlight - escaneia todos os comentários com todo (tarefas por ainda fazer no código) <https://marketplace.visualstudio.com/items?itemName=wayou.vscode-todo-highlight>
- vscode-icons - um grupo de ícones para ajudar nos achar mais rápido no visual dos arquivos e pastas do VSCode <https://marketplace.visualstudio.com/items?itemName=vscode-icons-team.vscode-icons>
- React Native Paper Snippets - um Intellisense com dicas para a lib de UI React Native Paper <https://marketplace.visualstudio.com/items?itemName=arpitBhalla.rnp-snippets>

3. Configure o VSCode para produtividade (recomendadas para https://code.visualstudio.com/Docs/languages/javascript#_code-actions-on-save)

3.1 Utilize um email institucional e **adicione o Copilot ao VSCode**;

3.2 **Configure o ESLint e o Prettier para produtividade**;

Um boa **qualidade de código começa** pela análise estática do código. Por isso, vamos utilizar Typescript + linters (fonte: <https://reactnative.dev/docs/testing-overview>).

Eu sugiro que vocês deem uma olhada nessa parte da documentação do React Native para entender mais das **recomendações de Q&A**: <https://reactnative.dev/docs/testing-overview>

Nota 1: **Linting** é o processo de verificar o código para problemas potenciais. É uma prática comum usar ferramentas de linting para detectar problemas no início do processo de desenvolvimento, conforme o código é escrito. Para realizar esse processo no nosso código de forma automatizada pelo VSCode, nós vamos precisar **instalar os dois plugins** ESLint (<https://marketplace.visualstudio.com/items?itemName=dbaeumer.vscode-eslint>) e Prettier (<https://marketplace.visualstudio.com/items?itemName=esbenp.prettier-vscode>) e, depois, fazer as configurações listadas abaixo:

- **ESLint** é um linter JavaScript que ajuda a encontrar e corrigir erros no seu código. É uma ótima ferramenta para ajudar a escrever um código melhor e detectar erros antes que eles cheguem à produção. Em conjunto, você pode usar o **Prettier**, um formatador de código que garante que todos os arquivos de código sigam um estilo consistente.(fonte: <https://docs.expo.dev/guides/using-eslint/>). Para o ESLint: **No VSCode busque por "formatOnSave" e marque "Editor: formatOnSave"**; Para o Prettier: **No VSCode busque por "defaultFormatter" e marque Editor: "Default Formatter"**.

Nota 2: Em resumo, nós utilizaremos o Prettier para questões de **formatação** de código e o ESLint para questões de **qualidade** de código (sintaxe).

4. Execute o projeto:

4.1 Faça o **setup para o ambiente**: <https://docs.expo.dev/get-started/set-up-your-environment/>

4.2 Execute o Expo: **npx expo start** <https://docs.expo.dev/get-started/start-developing/>

Dica: Utilize um **dispositivo físico, preferencialmente**. A execução do app é mais rápida em dispositivos físicos.

5. **Se atualizar sobre Git e GitHub** (indico a play list do Manginhos): youtube.com/playlist?list=PLfDIQS3HDSgk59zIzXhWvuvRQDIk2Kbak

6. **Crie um APK no perfil "development" com o serviço EAS**: <https://docs.expo.dev/build/setup/> .

O que é o React Native?

✓ Concluído

De que é composto o React Native?

✓ Concluído

Como configurar o ambiente para trabalhar com Expo

✓ Concluído

Como criar o primeiro app com Expo

✓ Concluído

✓ Semana 2 - Fundamentos de React, Interfaces Gráficas e Navegabilidade do App - 12/08/2026 e 14/08/2026

Em aula

- 🧑🧑🧑 **Definição do app que será desenvolvido.**
- Retirada de dúvidas sobre a instalação do ambiente
- Props e States <https://reactnative.dev/docs/intro-react#props> e <https://reactnative.dev/docs/intro-react#state>
- Debug de apps <https://docs.expo.dev/debugging/errors-and-warnings/> e <https://reactnative.dev/docs/react-native-devtools>
- Passando valores entre os componentes <https://reactnative.dev/docs/intro-react#custom-components>
- Ciclo de Vida dos Componentes <https://react.dev/learn/lifecycle-of-reactive-effects>
- Construindo interfaces gráficas com componentes nativos e externos <https://reactnative.dev/docs/components-and-apis> e <https://reactnativepaper.com/> (e <https://docs.expo.dev/develop/user-interface/splash-screen-and-app-icon/> , este link é o início da seção User Interface, leia os demais assuntos presentes nessa seção)
- Utilizando dependências da navegabilidade <https://docs.expo.dev/develop/file-based-routing/>
- Estrutura do projeto e extensões da IDE

Para casa

1. Construção do app de exemplo, tópicos:

- Props: vídeos 20 ao 22 e <https://reactnative.dev/docs/intro-react#props>
- Estrutura do projeto: https://github.com/vagnersilvaifsul/pdm_exemplo_expo.git
- States: 24 <https://reactnative.dev/docs/intro-react#state>
- Ciclo de Vida dos Componentes: 25 <https://react.dev/learn/lifecycle-of-reactive-effects>
- Construção de UI com core-components: 31 ao 41 <https://reactnative.dev/docs/components-and-apis> e <https://docs.expo.dev/develop/user-interface/splash-screen-and-app-icon/> (toda seção user-interface)

2. Estudo documentação oficial do RN:

- Components, suas props e states: <https://reactnative.dev/docs/intro-react>

Algumas Palavras sobre React **Hooks**:

- Um Hook só pode ser chamado no nível superior de um componente de função. Então, um Hook não pode ser chamado em um loop ou aninhado em uma função;
- Um Hook não pode ser chamado condicionalmente (não pode estar dentro de if-else).
- Um Hook só pode ser usado em componentes de função (o que estamos utilizando) e não em componentes de classe (forma antiga de usar React);
- Um Hook deve iniciar com o prefixo use (uma convenção);
- É possível criarmos nossos próprios Hooks;
- Nós utilizaremos esses React Hooks: useState, useEffect, useRef e useContext;
- Para saber mais sobre React Hooks: <https://pt-br.react.dev/reference/react/hooks#state-hooks>

Apps escolhidos pelos alunos:

Leandro: Projeto

Lucas Bach: Projeto

Marcelo : Projeto

Nícollas: Projeto

Rodrigo: Projeto

Rômulo: Projeto

Vinícius Borges : Projeto

Fundamentos de React

✓ Concluído

Seção Design da documentação do RN

✓ Concluído

Playground para aprender Flexbox

✓ Concluído

API de componentes do React Native

✓ Concluído

✓ Semana 3 - Funcionalidade SignIn, SignUP, CRUD do Perfil do usuário, e afins - 19/08/2026 e 21/08/2026

Em aula

Por assunto:

- Retirada de dúvidas sobre props e states, ciclo de vida dos componentes, debug, navegabilidade, e interfaces gráficas com componentes nativos
- Instalação das dependências do Firebase. Configuração do Firebase <https://docs.expo.dev/guides/using-firebase/#using-firebase-js-sdk>
- Inclusão do serviço Authentication <https://firebase.google.com/docs/auth/web/start?hl=pt-br>
- API de Context <https://react.dev/learn/passing-data-deeply-with-context> <https://react.dev/reference/react/useContext>
- Navegabilidade do app <https://docs.expo.dev/router/introduction/>
- Componentes de interface gráfica que dão suporte ao serviço authentication <https://reactnativepaper.com/> (uma lib com os componentes pré-tematizados/estilizados) e <https://reactnative.dev/docs/components-and-apis> (as props são herdadas destes

componentes pela Native Paper)

- Ícones através da lib react-native-vector-icons <https://oblador.github.io/react-native-vector-icons/>
- Tema do app <https://callstack.github.io/react-native-paper/docs/guides/theming> ou, se você quer ver como é implementado em um projeto, acesse: https://github.com/vagnersilvaifsul/pdm_exemplo/blob/thema_personalizado/src/navigation/Providers.tsx

Por tarefa:

- Entrar (logar no app): https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo1_entrar
- Sair (sair do app): https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo1_sair

Para casa

1. Entrar (logar no app): https://github.com/vagnersilvaifsul/pdm_exemplo_expo/commits/modulo1_entrar/
2. Sair (sair do app): https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo1_sair
3. Lembre! **Avance** em cada funcionalidade **analisando a sequencia de commits** do projeto de exemplo

- ✓ Semana 4 - Funcionalidade SignIn, SignUP, CRUD do Perfil do usuário, e afins - 26/08/2026 e 28/08/2026

Em aula

- Formulários com React Hook Form <https://www.react-hook-form.com/> e <https://www.npmjs.com/package/@hookform/resolvers>
- Validação de formulários com Yup + Hook Form <https://www.npmjs.com/package/yup>

Para casa

Entrar (logar no app, utilizando Hook Form e Yup): https://github.com/vagnersilvaifsul/pdm_exemplo_expo/commit/f6dd428f719df359783edf58628c13ff6dd8fac0

✓ usuário, e afins - 02/09/2026 e 04/09/2026

Em aula

- Preload https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo1_preload
- Automatização do login com Secure Store <https://docs.expo.dev/versions/latest/sdk/securestore/>
- Sair https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo1_sair
- SignUp https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo1_signup
- Firestore
 - Introdução: <https://firebase.google.com/docs/firestore?hl=pt-br>
 - Escolher uma estrutura de dados: <https://firebase.google.com/docs/firestore/manage-data/structure-data?hl=pt-br>
 - Adicionar dados ao Cloud: <https://firebase.google.com/docs/firestore/manage-data/add-data?hl=pt-br>
 - Receber dados com o Cloud: <https://firebase.google.com/docs/firestore/query-data/get-data?hl=pt-br>
- Finalizar o Módulo 1

Para casa

Finalizar o Módulo 1:

1. Preload: https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo1_preload
2. Sair https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo1_sair
3. SignUp https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo1_signup
4. CRUD do Perfil do Usuario: https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo1_perfil

Diagrama de Classes do App de Exemplo

Marcar como feito

✓ Semana 6 - Funcionalidade SignIn, SignUp, CRUD do Perfil do usuário, e afins - 09/09/2026 e 11/09/2026

Em aula

- Entrega e defesa do **Módulo 1** para **avaliação** - Funcionalidade SignIn, SignUp, CRUD de Perfil do usuário, e afins

Para casa

1. Entrega e defesa do **Módulo 1** para **reavaliação**, funcionalidade SignIn, SignUp CRUD do Perfil do Usuário, e afins (agendar a defesa com o professor)

ESPECIFICAÇÃO DO MÓDULO 1

Você deverá apresentar e defender a implementação da **autenticação do app**, bem como, o **CRUD do perfil do usuário**, ambos utilizando os serviços, Authentication e Firestore Database do Firebase, respectivamente. O software a ser defendido deve seguir a arquitetura de software apresentada em sala de aula, incluindo as pastas, assets, components, context, navigation e screens. A construção das interfaces gráficas é livre, isto é, você pode utilizar componentes nativos, styled-componentes, React Native Elements, ou outra biblioteca React.

Você ainda deverá observar as seguintes condições:

1. Utilizar a API de Context e o SDK do Firebase;
2. Navegabilidade por BottomTab;
3. As operações CRUD devem envolver: Create, Read, Update, e Hard Delete.

Esta é uma lista das **funcionalidades que você deve apresentar e defender**:

1. **SignIn** (completo) pelo email e senha https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo1_entrar (veja também a verificação de validação de email adicionada em https://github.com/vagnersilvaifsul/pdm_exemplo_expo/commit/962c091718726daa5a7fb7691a08e8bd556ddf0f ;
2. **SignOut** https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo1_sair
3. **SignUp** (completo) https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo1_signup
4. **Confirmação de email** https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo1_signup
5. **Preload com SignIn automatizado** pelo cache (via SecureStore) https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo1_preload
6. **CRUD do perfil do usuário** https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo1_perfil
7. **Recuperação de senha** Você deve desenvolver a partir do conhecimento adquirido nas aulas

8. **Navegabilidade do app** (está em todas as branches, utilize a branch main, onde é feita a junção das funcionalidades do app, em https://github.com/vagnersilvaifsul/pdm_exemplo_expo.git)

Entregue aqui o link para o repositório do seu projeto no github

✓ Concluído

Aberto: quarta-feira, 1 jul. 2026, 00:00

Vencimento: sexta-feira, 18 set. 2026, 23:59

✓ Semana 7 - Funcionalidade Upload de imagens e Build do App - 16/09/2026 e 18/09/2026

Em aula

- Crie sua primeira versão do app (Build do app): <https://docs.expo.dev/build/setup/>
- Gestione seus apps através do Expo Orbit: <https://expo.dev/orbit>
- Upload de Imagens no serviço Storage do Firebase <https://firebase.google.com/docs/storage?hl=pt-br> e <https://firebase.google.com/docs/storage/web/start?hl=pt-br> e <https://firebase.google.com/docs/storage/web/upload-files?hl=pt-br>
- expo-image-picker <https://docs.expo.dev/versions/latest/sdk/imagepicker/>
- expo-image-manipulator <https://docs.expo.dev/versions/latest/sdk/imagemanipulator/>
- Fazer o Módulo 2 - Funcionalidade Upload de imagens para o Perfil do usuário https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo2_upload

Para casa

1. Fazer o Módulo 2 - Funcionalidade Upload de imagens para o Perfil do usuário. https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo2_upload e <https://firebase.google.com/docs/storage/web/start?hl=pt-br>

✓ Semana 8 - Funcionalidade Upload de imagens - 23/09/2026 e 25/09/2026

Em aula

- Retirada de dúvidas sobre Upload de Imagens no serviço Storage do Firebase
- Finalizar o Módulo 2 - Funcionalidade Upload de imagens https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo2_upload

Para casa

1. Finalizar o Módulo 2 - Funcionalidade Upload de imagens. https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo2_upload

✓ Semana 9 - Funcionalidade Upload de imagens - 30/09/2026 e 02/10/2026

Em aula

Entrega e defesa do **Módulo 2** para **avaliação**, funcionalidade Upload de imagens.

Para casa

1. Resolver pendências e finalizar o **Módulo 2** para **reavaliação**, funcionalidade Upload de Imagens. (agendar a defesa com o professor)

ESPECIFICAÇÃO DO MÓDULO 2

Regras para entrega:

1. Você deve buscar a imagem no app nativo de Galeria de Imagens ou permitir que o usuário possa tirar uma foto a partir do seu app;
2. A imagem adquirida pelo seu app na regra 1 deve ser tratada pelo seu app. Isto inclui, comprimir e redimensionar a imagem, utilizando a lib expo-image-manipulator, com qualidade máxima de "80%", utilizando compressor "PNG", nas dimensões 150x150 (ou uma que se adeque a informação da sua imagem);
3. A imagem deve ser enviada para o serviço de Storage do Firebase, na pasta imagens, com um nome semântico (que faça sentido para a realidade do seu app e facilite sua localização no dashboard do Storage);
4. A URL para a imagem no serviço de Storage deve ser armazenada no serviço de banco de dados Firestore;
5. O app deve ser capaz de baixar a imagem do serviço de Storage para exibí-la ao usuário do app.

Semana 10 - Funcionalidade CRUD via SDK do Firestore Web -

✓ 07/10/2026 e 09/10/2026

Em aula

CRUD via SDK do SDK do Firestore Web <https://firebase.google.com/docs/firestore?hl=pt-br> (no menu da referência, à esquerda, você pode navegar por toda documentação para ver como: modelar o Firestore, administrar, e realizar o CRUD)

Fazer o Módulo 3 – Funcionalidade CRUD via SDK do Firestore Web https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo3_crud_sdk_firebase

Para Casa

1. Fazer o Módulo 3 – Funcionalidade CRUD via SDK do Firestore Web https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo3_crud_sdk_firebas

✓ Semana 11 - Funcionalidade CRUD via SDK do Firestore Web - 14/10/2026 e 16/10/2026

Em aula

Retirada de dúvidas sobre CRUD via SDK do Firestore Web

Finalizar o Módulo 3 - Funcionalidade CRUD via SDK do Firestore Web https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo3_crud_sdk_firebase

Para casa

Finalizar o Módulo 3 - Funcionalidade CRUD via SDK do Firestore Web https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo3_crud_sdk_firebase

✓ Semana 12 - Funcionalidade CRUD via SDK do Firestore Web - 21/10/2026 e 23/10/2026

Em aula

- Entrega e defesa do Módulo 3 para avaliação, funcionalidade CRUD via SDK do Firestore Web

Para casa

Resolver pendências e finalizar o **Módulo 3** para **reavaliação**, Funcionalidade CRUD via SDK do Firestore Web (agendar com o professor)

ESPECIFICAÇÃO DO MÓDULO 3

Você deverá apresentar e defender a implementação de outro CRUD, baseado na sua proposta de app e considerando o que foi desenvolvido nas aulas (ou videoaulas) sobre CRUD com SDK do Firestore Web. Você ainda deverá observar as seguintes condições:

Utilizar a API de Context (<https://pt-br.reactjs.org/docs/context.html#when-to-use-context/>) e as funções da documentação do SDK do Firestore Web.

As operações CRUD devem envolver: Create, Read (listar todos os documentos e filtrar documentos por uma chave de pesquisa), Update, e Delete.

✓ Semana 13 - Funcionalidade Notificações - 28/10/2026 e 30/10/2026

Em aula

- Noções Básicas: <https://docs.expo.dev/push-notifications/what-you-need-to-know/>
- Expo Notifications <https://docs.expo.dev/versions/latest/sdk/notifications/>
- In App Notifications <https://docs.expo.dev/versions/latest/sdk/notifications/#schedulesnotificationasyncrequest>
- Expo Device <https://docs.expo.dev/versions/latest/sdk/device/>
- Expo Go vs Development Builds: Qual você deve usar? <https://expo.dev/blog/expo-go-vs-development-builds>
- Envio de notificações pelo Firebase <https://docs.expo.dev/push-notifications/fcm-credentials/> e <https://console.firebase.google.com/>
- API da Expo Notifications <https://docs.expo.dev/versions/latest/sdk/notifications/>
- Fazer o Módulo 4 para avaliação, funcionalidade Notificações https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo4_notificacoes# e <https://github.com/>

[vagnersilvaifsul/pdm_exemplo_expo/tree/modulo4_notificacoes_in_app](https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo4_notificacoes_in_app) e https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo4_notificacoes_fcm_token

Para casa

1. Estudar a documentação sobre Cloud Messaging com Expo, em: <https://docs.expo.dev/versions/latest/sdk/notifications/>.
2. Fazer o **Módulo 4** para **avaliação**, funcionalidade Notificações. https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo4_notificacoes e https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo4_notificacoes_in_app e https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo4_notificacoes_fcm_token

✓ Semana 14 - Funcionalidade Notificações - 04/11/2026 e 06/11/2026

Em aula

Finalizar o Módulo 4 para avaliação, funcionalidade Notificações https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo4_notificacoes e https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo4_notificacoes_in_app e https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo4_notificacoes_fcm_token

Para casa

Finalizar o Módulo 4 para avaliação, funcionalidade Notificações https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo4_notificacoes e https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo4_notificacoes_in_app e https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo4_notificacoes_fcm_token

Roteiro para Criar um Projeto Firebase CLI

1. Instale o Firebase CLI no seu ambiente, execute **npm install -g firebase-tools** <https://>

firebase.google.com/docs/cli?hl=pt-br#windows-npm

2. Faça o login no Firebase, execute **firebase login** (e siga os passos necessários no navegador web) <https://firebase.google.com/docs/cli?hl=pt-br#sign-in-test-cli>
3. Crie uma pasta para o projeto Firebase CLI, no contexto do professor ele criou ~/VSCode/expo/FirebaseCLI/pdm aulas_expo (na pasta pdm aulas_expo será criado o projeto Firebase CLI)
4. Abra um terminal na pasta de trabalho e execute **firebase init** <https://firebase.google.com/docs/cli?hl=pt-br#windows-npm>
5. Selecione as funcionalidades que você deseja ter no projeto (gerenciar através do projeto). No contexto do professor foram selecionados os serviços: Firestore, Functions, Storage, Emulators (para emular os serviços do Firebase localmente) e Extensions
6. No passo seguinte selecione: *Use an existing project*
7. Na lista exibida pelo CLI selecione o projeto que você deseja gerenciar pelo projeto que você está criando com o Firebase CLI
8. Na sequência o Firebase CLI vai perguntar quais nomes você deseja dar aos arquivos de configuração dos serviços, o professor manteve os nomes sugeridos pelo Firebase CLI
9. Selecione a linguagem que você pretende trabalhar, o professor escolheu Javascript para ser compatível com o projeto que está público no Github https://github.com/vagnersilvaifsul/pdm_aulas_firebase_cli
10. Selecione utilizar o ESLint
11. Selecione instalar as dependências do projeto (ou deixa para fazer isso mais tarde)
12. Siga selecionando o nome dos arquivos para configuração dos serviços
13. Selecione os serviços que você deseja emular localmente (se você desejar emular o Firebase em sua máquina. Caso você não deseje, não selecione serviços nessa etapa) <https://firebase.google.com/docs/emulator-suite?hl=pt-br> e https://firebase.google.com/docs/emulator-suite/connect_and_prototype?hl=pt-br#prototype_interactively
14. Selecione as portas para os serviços que serão emulados (se você optou por criar emuladores). O professor manteve as portas indicadas pelo Firebase CLI
15. Realizadas as etapas acima, seu projeto estará pronto para ser aberto no VSCode. E você poderá personalizar cada serviço e a segurança do seu projeto. Na aula o professor focou nos **Cloud Functions** para demonstrar como gerar notificações de forma automatizada e como implantá-las no Firebase. <https://firebase.google.com/docs/functions?hl=pt-br> e <https://firebase.google.com/docs/functions/manage-functions?hl=pt-br&gen=2nd#deploy-functions>

Quer escrever, testar e implantar as suas próprias funções Firebase FCM. Consulte a documentação a seguir?

1. Sobre as Mensagens do FCM https://firebase.google.com/docs/cloud-messaging/concept-options?hl=pt-br#notifications_and_data_messages
2. Visão Geral da Arquitetura do FCM <https://firebase.google.com/docs/cloud-messaging/fcm-architecture?hl=pt-br>
3. Sobre os Tipos de Mensagens do FCM <https://firebase.google.com/docs/cloud-messaging/concept-options?hl=pt-br>
4. Fazer a assinatura do app cliente em um tópico (😞 infelizmente o Expo não suporta essa funcionalidade, teríamos que ejetar de Expo para CLI) https://firebase.google.com/docs/cloud-messaging/android/send-multiple?hl=pt-br#subscribe_the_client_app_to_a_topic . A alternativa seria fazer a segmentação na Cloud Function baseado em <https://firebase.google.com/docs/cloud-messaging/js/send-multiple?hl=pt-br> e no App baseado em: <https://docs.expo.dev/versions/latest/sdk/notifications/#getdevicepushtokenasync>
5. Como escrever funções acionadas pelo Cloud Firestore <https://firebase.google.com/docs/functions/firestore-events?hl=pt-br&gen=2nd>
6. Executar funções localmente para testar as funções antes da implantação no FCM <https://firebase.google.com/docs/functions/local-emulator?hl=pt-br>
7. Implantar funções no FCM <https://firebase.google.com/docs/functions/manage-functions?hl=pt-br&gen=2nd#deploy-functions>

Exemplo de Projeto Firebase CLI

Marcar como feito

Instalação e utilização do Firebase CLI

Marcar como feito

Introdução ao conjunto de emuladores locais do Firebase

Marcar como feito

Conectar o app e começar a prototipagem

Marcar como feito

Cloud Functions

Marcar como feito

Implantar Cloud Functions

Marcar como feito

Agendamento de atividades com cron (cron to schedule jobs)

Marcar como feito

✓ Semana 15 - Funcionalidade Notificações - 11/11/2026 e 13/11/2026

Em aula

Entrega e defesa do **Módulo 4** para **avaliação**, funcionalidade Notificações

Para casa

Resolver pendências e finalizar o **Módulo 4** para **reavaliação**, funcionalidade Notificações (agendar com o professor)

ESPECIFICAÇÃO DO MÓDULO 4

Você deverá apresentar e defender a implementação do serviço de **Cloud Notifications** e **In App** baseado no que foi desenvolvido em sala de aula. Você deverá observar as seguintes exigências:

- O app deve receber e processar notificações do **serviço FCM** (*Firebase Cloud Messaging*), nessas situações:
 1. Com o app fechado (receber a notificação e ao "clicar" nela encaminha para a rota correta do app);
 2. Com o app parado (receber a notificação e ao "clicar" nela encaminha para a rota correta do app);
 3. Com o app aberto (receber a notificação e alertar o usuário de seu recebimento, depois, se o usuário decidir, encaminhar para a rota correta do app).
 4. As rotas do app você irá definir a partir de perfis de acesso as notificações, como, o exemplo apresentado em aula, empresas e usuarios (se baseie no exemplo, mas faça a solução para o contexto do seu app).
- As mensagens devem ser enviadas através do **FCM** (dashboard de FCM (manual) ou *Cloud Functions* (automatizado)) com as seguintes segmentações:
 1. **Para todos os dispositivos** que tem o app instalado e com a rota definida para abrir o app na página correta.
- O app deve receber e processar notificações **In App**, nessas situações:

1. Com o app fechado (receber a notificação e ao "clicar" abrir o app na rota padrão);
2. Com o app parado (receber a notificação e ao "clicar" abrir o app na rota padrão);
3. Com o app aberto (receber a notificação e ao "clicar" abrir o app na rota padrão);
4. O agendamento (schedule) da notificação não deve ter o mesmo padrão do exemplo.
Elabore outro esquema de agendamento, diferente do exemplo.

- As mensagens *In App* devem ser envidas através da biblioteca **Expo Notifications** (expo-notifications):

1. **Para o dispositivo** que tem o app instalado.

Entregue aqui o seu APK (em modo Preview ou Production) com a funcionalidade de Notificações

Abre: domingo, 20 dez. 2026, 00:00 **Vencimento:** domingo, 27 dez. 2026, 23:59

✓ Semana 16 - Funcionalidade Mapas e Localização - 18/11/2026 e 20/11/2026(feriado)

Em aula

Mapas e localização <https://docs.expo.dev/versions/latest/sdk/map-view/>

Marcadores <https://github.com/react-native-maps/react-native-maps?tab=readme-ov-file>
e <https://github.com/react-native-maps/react-native-maps/blob/master/docs/marker.md>

Fazer o Módulo 5, funcionalidade Mapas e localização https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo5_mapas

Para casa

Fazer o Módulo 5, funcionalidade Mapas e localização https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo5_mapas

✓ Semana 17 - Funcionalidade Mapas e Localização - 25/11/2026 e 27/11/2026

Em aula

Mapas e localização

Marcadores

Finalizar o Módulo 5, funcionalidade Mapas e localização https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo5_mapas

Para casa

Finalizar o Módulo 5, funcionalidade Mapas e localização https://github.com/vagnersilvaifsul/pdm_exemplo_expo/tree/modulo5_mapas

✓ Semana 18 - Funcionalidade Mapas e Localização - 02/12/2026 e 04/12/2026

Em aula

Entrega e defesa do **Módulo 5** para **avaliação**, funcionalidade Mapas e localização

Para casa

Resolver pendências e finalizar o Módulo 5 para reavaliação, funcionalidade Mapas e localização

ESPECIFICAÇÃO DO MÓDULO 5

Você deverá apresentar e defender a implementação de Mapas e Localização baseado no que foi desenvolvido em sala de aula. Você deverá observar as seguintes exigências:

1. O mapa deve inicializar no prédio do IFSul - Câmpus Pelotas;
2. O mapa deve apresentar, no mínimo, três marcadores, próximos a localização inicial do mapa (ao lado do prédio do câmpus ou em frente);
3. Ao clicar no marcador deve, no mínimo, ser possível ver detalhes sobre esse marcador;
4. Ao inserir um novo documento na base de dados (no exemplo foi utilizado empresa, você vai trabalhar no contexto do seu app, não utilizar o exemplo para a defesa), o app deve validar o endereço pelo CEP e, ao salvar (como no exemplo) ele deve buscar as coordenadas geográficas do endereço, em decimal, e salvar no documento;
5. Um novo documento também deve ser exibido como um novo marcador no mapa, seguindo as regras apresentadas nos itens anteriores.

✓ Semana 19 - Funcionalidade Mapas e Localização - 09/12/2026 e 11/12/2026

Em aula

Entrega e defesa do **Módulo 5** para **reavaliação**, funcionalidade Mapas e localização

✓ Semana 20 e 21 - Entrega de Notas e Retirada de Dúvidas - 16/12/2026, 18/12/2026 e 23/12/2026

Em aula

Entrega de notas e retirada de dúvidas